SQL 3

Gage Powell
ISIN 325
April 6, 2026
Dr. Greg Gogolin

Table Of Contents

Page 1…….……Cover Page

Page 2……...……Table of Contents

Page 3…….….....Query 1 - 4

Page 4…….....Query 5 & 6

Page 5…….…….…….....Query 7 - 10

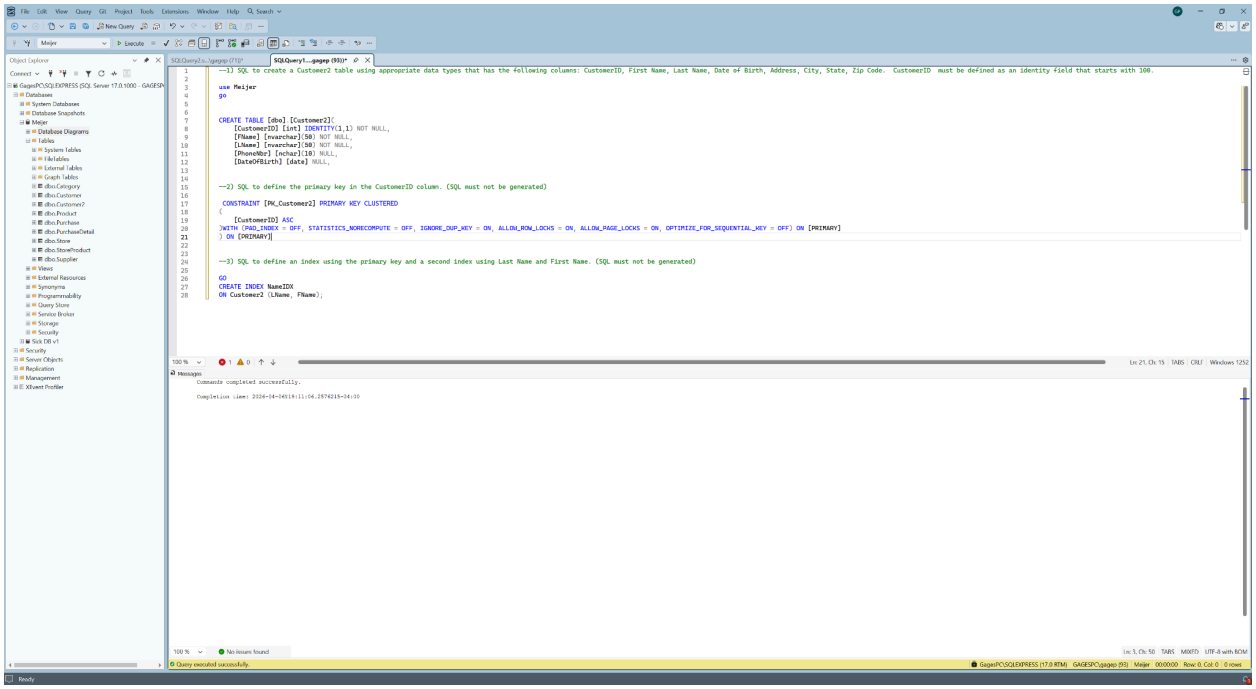
Page 6…….….…….....Query 11 & 12

Page 7…….……..…….....Query 13 & 14

Page 8……..….……Query 15 & 16

Page 9……Query 17 & 18

Query 1 - 3

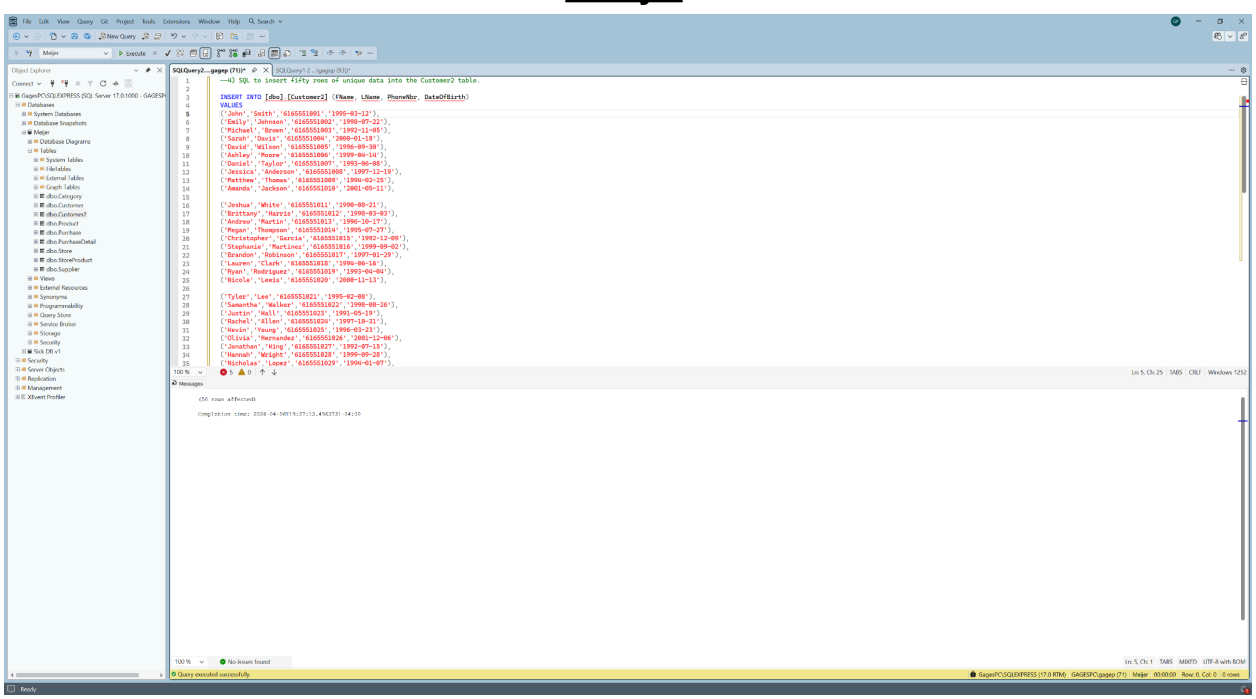


```
--1) SQL to create a Customer2 table using appropriate data types that has the following columns: CustomerID, First Name, Last Name, Date of Birth, Address, City, State, Zip Code.  CustomerID  must be defined as an identity field that starts with 100.

use Meijer
go


CREATE TABLE [dbo] [Customer2](
    [CustomerID] [int] IDENTITY(1,1) NOT NULL,
    [FName] [nvarchar](50) NOT NULL,
    [LName] [nvarchar](50) NOT NULL,
    [PhoneNbr] [nchar](10) NULL,
    [DateOfBirth] [date] NULL,


--2) SQL to define the primary key in the CustomerID column. (SQL must not be generated)

 CONSTRAINT [PK_Customer2] PRIMARY KEY CLUSTERED
 (
     [CustomerID] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = ON, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON, OPTIMIZE_FOR_SEQUENTIAL_KEY = OFF) ON [PRIMARY]
) ON [PRIMARY]


--3) SQL to define an index using the primary key and a second index using Last Name and First Name. (SQL must not be generated)

GO
CREATE INDEX NameIDX
ON Customer2 (LName, FName);
```

```
Commands completed successfully.

Completion time: 2024-04-04T19:11:04.2574235-04:00
```

Comments: These queries create the Customer2 table with appropriate data types, define the primary key, and create indexes to improve database performance. The purpose is to demonstrate how Data Definition Language is used to build and structure database objects efficiently.

Query 4

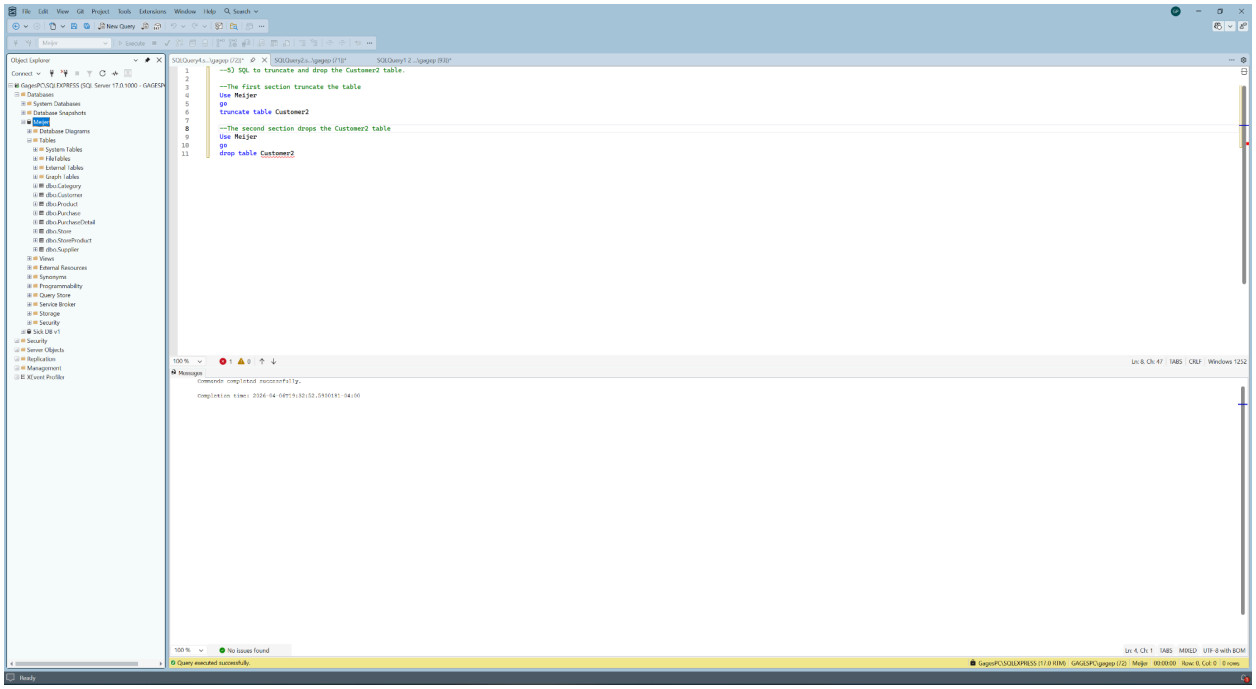


```
--4) SQL to insert fifty rows of unique data into the Customer2 table.

INSERT INTO [dbo].[Customer2] (FName, LName, PhoneNbr, DateOfBirth)
VALUES
('John','Smith','6165551001','1995-03-12'),
('Emily','Johnson','6165551002','1998-07-22'),
('Michael','Brown','6165551003','1992-11-05'),
('Sarah','Davis','6165551004','2000-01-18'),
('David','Wilson','6165551005','1996-09-30'),
('Ashley','Moore','6165551006','1999-04-14'),
('Daniel','Taylor','6165551007','1993-06-08'),
('Jessica','Anderson','6165551008','1997-12-19'),
('Matthew','Thomas','6165551009','1994-02-23'),
('Amanda','Jackson','6165551010','2001-05-11'),

('Joshua','White','6165551011','1998-08-21'),
('Brittany','Harris','6165551012','1998-03-03'),
('Andrew','Martin','6165551013','1996-10-27'),
('Megan','Thompson','6165551014','1995-07-27'),
('Christopher','Garcia','6165551015','1992-12-09'),
('Stephanie','Martinez','6165551016','1999-09-02'),
('Brandon','Robinson','6165551017','1997-01-29'),
('Lauren','Clark','6165551018','1994-06-16'),
('Ryan','Rodriguez','6165551019','1993-04-04'),
('Nicole','Lewis','6165551020','2000-11-13'),

('Tyler','Lee','6165551021','1995-02-08'),
('Samantha','Walker','6165551022','1998-08-26'),
('Justin','Hall','6165551023','1991-05-19'),
('Rachel','Allen','6165551024','1997-10-31'),
('Kevin','Young','6165551025','1996-03-23'),
('Olivia','Hernandez','6165551026','2001-12-06'),
('Jonathan','King','6165551027','1992-07-15'),
('Hannah','Wright','6165551028','1999-09-28'),
('Nicholas','Lopez','6165551029','1994-01-07'),
```

```
(50 rows affected)

Completion time: 2024-04-04T19:27:13.4563731-04:00
```

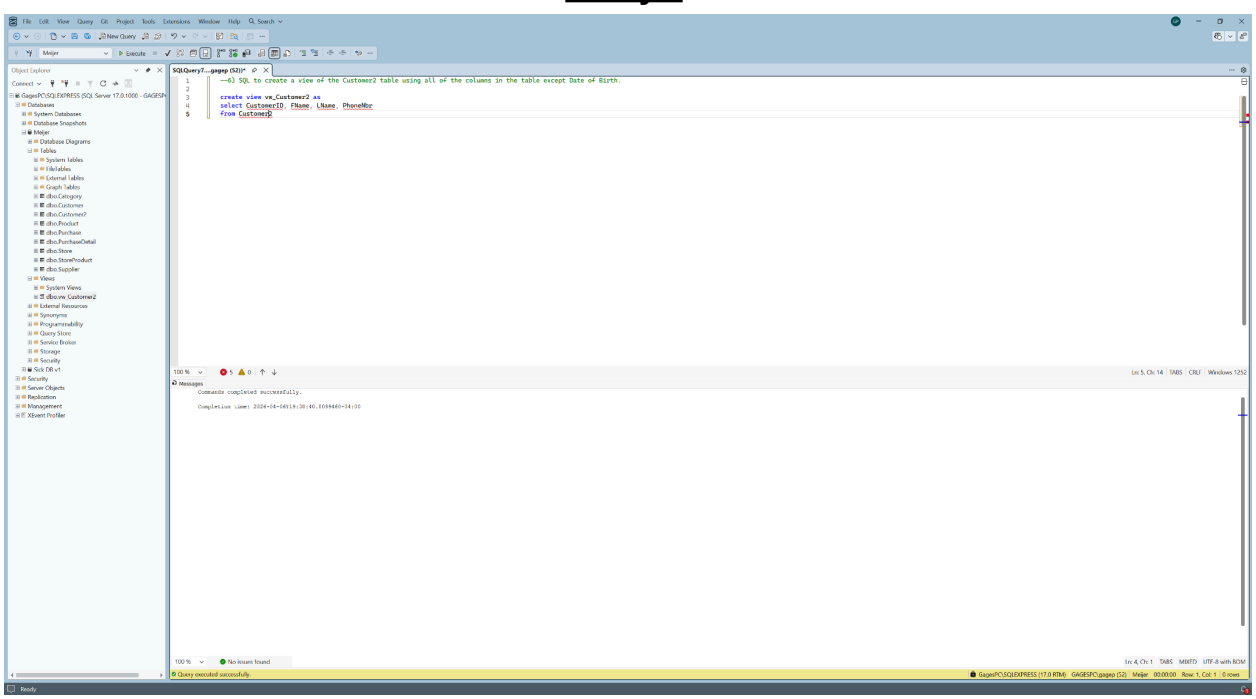
Comments: This query inserts fifty unique customer records into the Customer2 table to populate the database with sample data. The goal is to demonstrate how SQL insert statements are used to load and manage table data.

Query 5



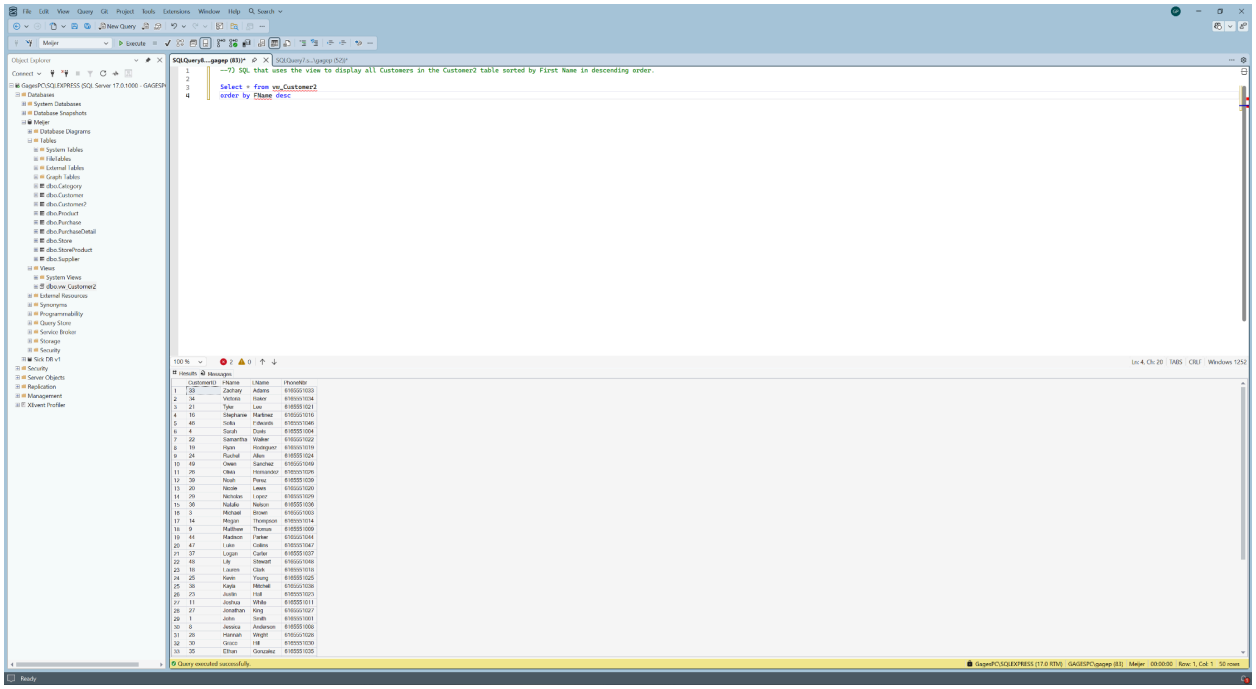
Comments: This query truncates and drops the Customer2 table to remove all data and delete the table structure from the database. This demonstrates how SQL can be used to safely remove database objects when they are no longer needed.

Query 6



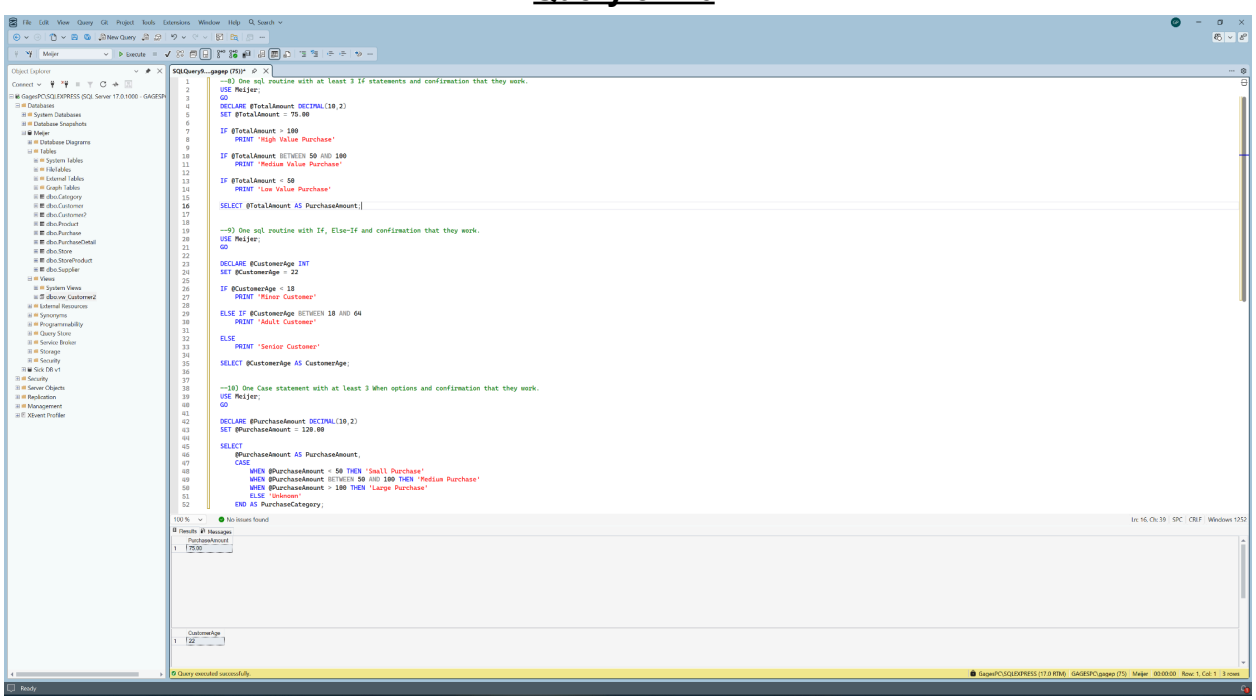
Comments: This query creates a view that displays all Customer2 table columns except Date of Birth. The purpose is to show how views can be used to limit sensitive data and simplify access to important information.

Query 7



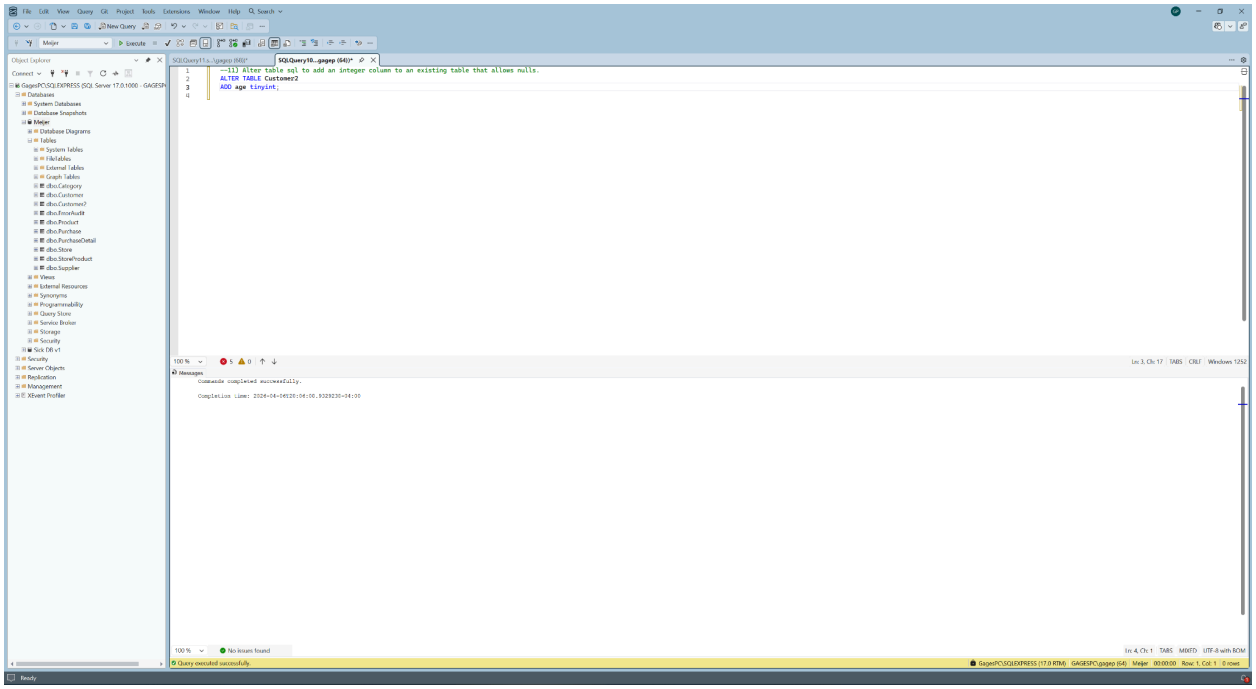
Comments: This query uses the view to display all customers sorted by first name in descending order. This demonstrates how views can be used like tables to retrieve and organize data.

Query 8 - 10



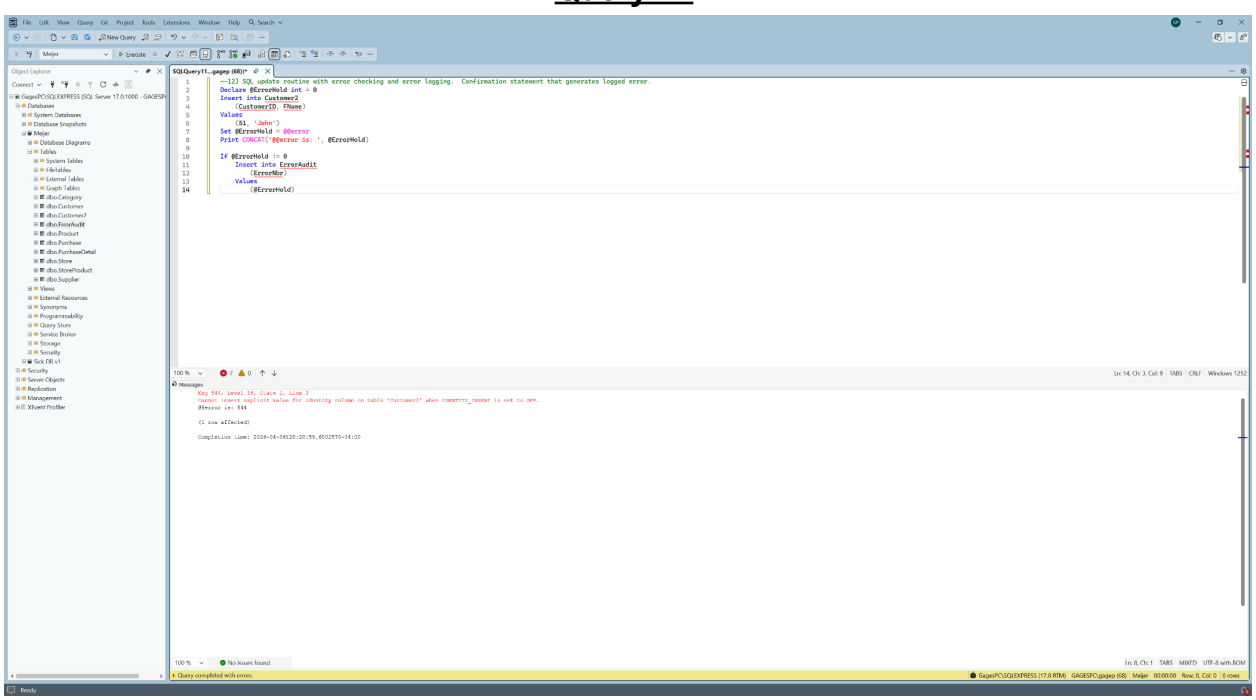
Comments: These queries demonstrate conditional logic in SQL using IF statements, IF-ELSE logic, and CASE statements. The purpose is to show how SQL can make decisions and categorize data based on conditions within the Meijer database.

Query 11



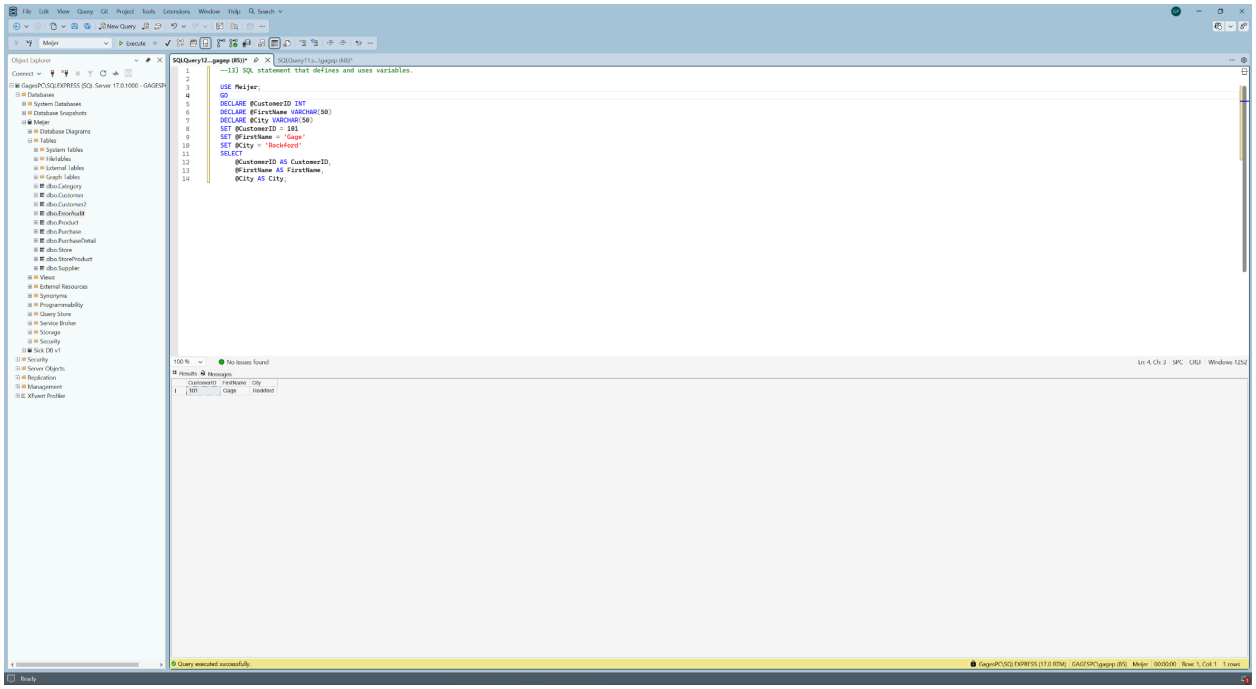
Comments: This query alters an existing table by adding a new integer column that allows null values. The goal is to demonstrate how SQL can modify table structures without recreating the table.

Query 12



Comments: This query performs an update routine with error checking and logs any errors into an error log table. The purpose is to demonstrate how SQL can track and record errors to improve database reliability and troubleshooting.

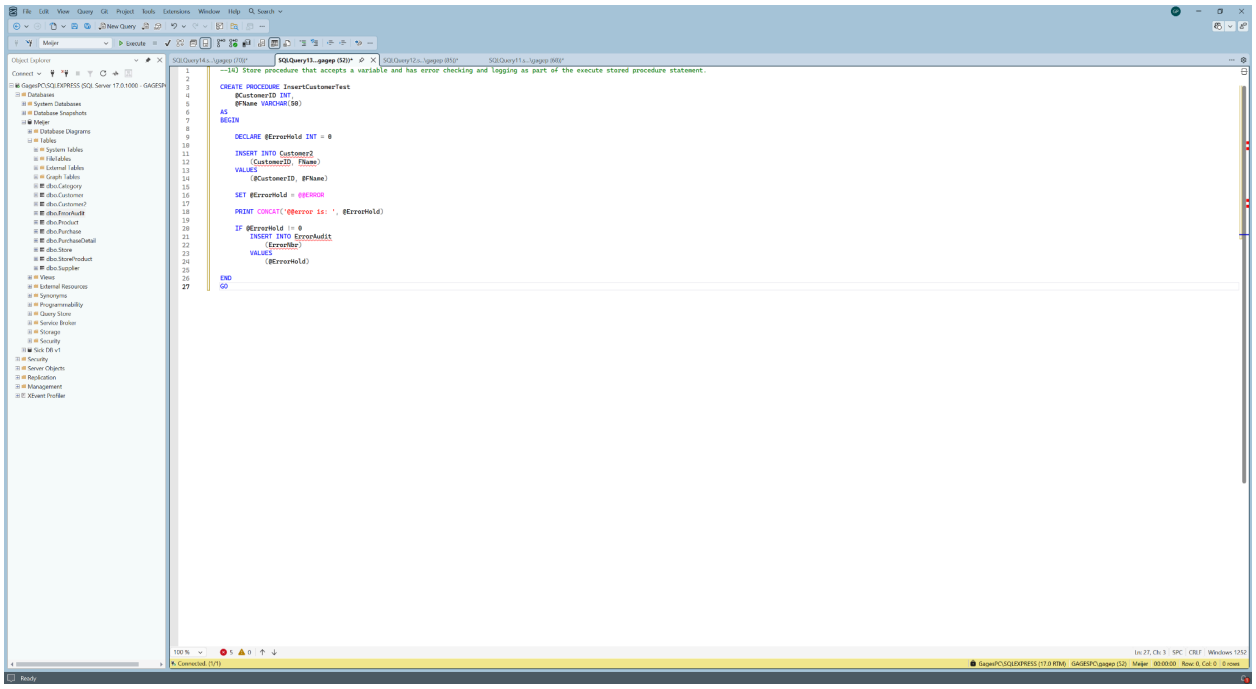
Query 13



```
--13) SQL statement that defines and uses variables.

USE Meijer;
GO
DECLARE @CustomerID INT
DECLARE @FirstName VARCHAR(50)
DECLARE @City VARCHAR(50)
SET @CustomerID = 101
SET @FirstName = 'Gage'
SET @City = 'Rockford'
SELECT
    @CustomerID AS CustomerID,
    @FirstName AS FirstName,
    @City AS City;
```

Comments: This query defines and uses SQL variables to store and display data values. The goal is to show how variables can be used to temporarily hold and manipulate data within SQL routines**.**

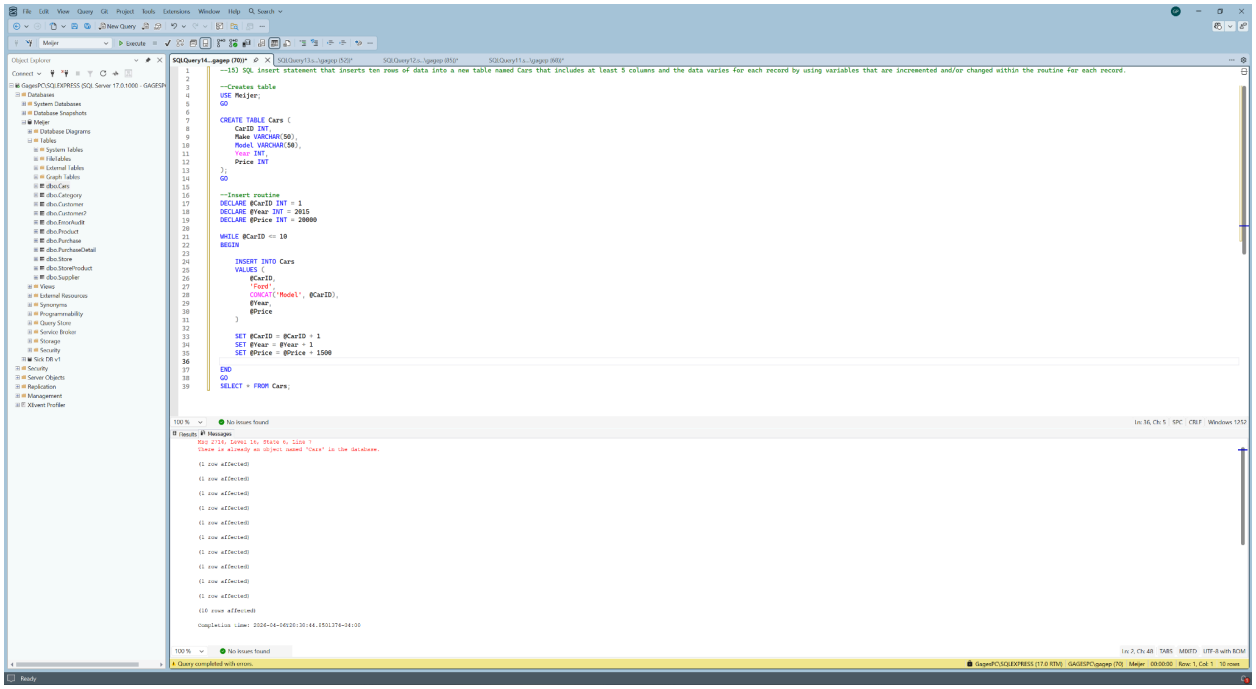
Query 14



```
--14) Store procedure that accepts a variable and has error checking and logging as part of the execute stored procedure statement.

CREATE PROCEDURE InsertCustomerTest
    @CustomerID INT,
    @FName VARCHAR(50)
AS
BEGIN

    DECLARE @ErrorHold INT = 0

    INSERT INTO Customer2
        (CustomerID, FName)
    VALUES
        (@CustomerID, @FName)

    SET @ErrorHold = @@ERROR

    PRINT CONCAT('@@error is: ', @ErrorHold)

    IF @ErrorHold != 0
        INSERT INTO ErrorAudit
            (ErrorNbr)
        VALUES
            (@ErrorHold)

END
GO
```

Comments: This stored procedure accepts variables as input and performs error checking and logging using @@ERROR. The purpose is to demonstrate how stored procedures automate database operations while tracking and recording errors.

Query 15

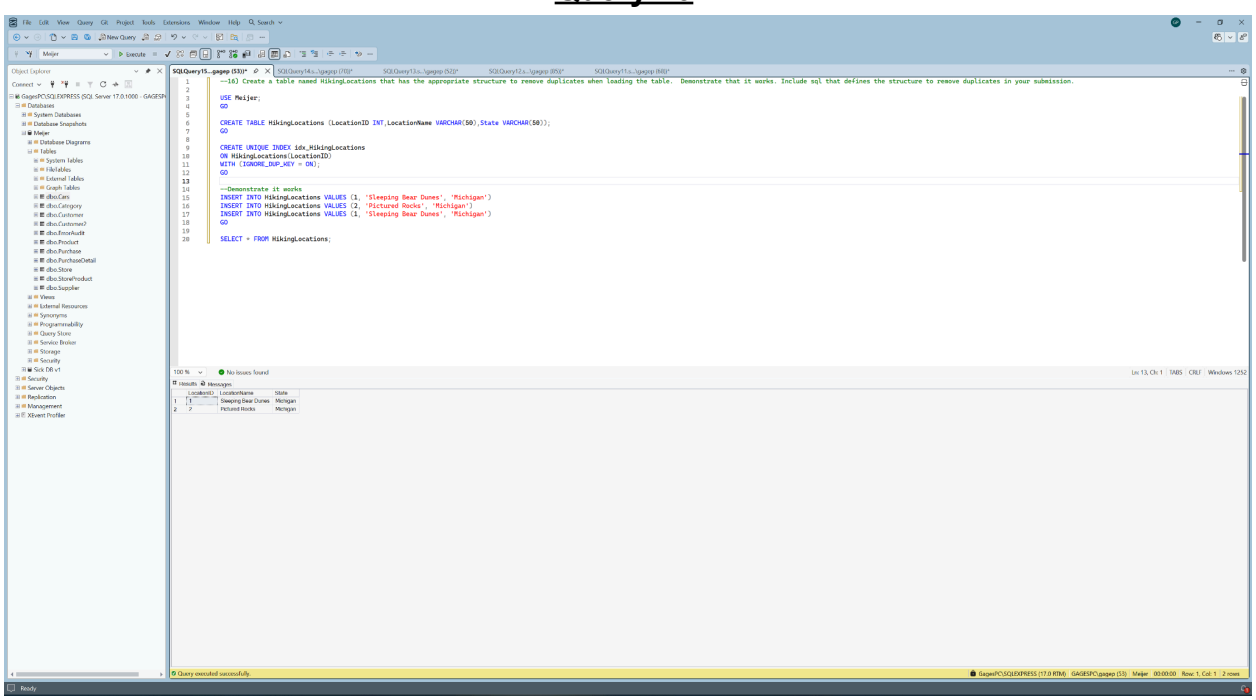


```
--15) SQL insert statement that inserts ten rows of data into a new table named Cars that includes at least 5 columns and the data varies for each record by using variables that are incremented and/or changed within the routine for each record.

--Creates table
USE Meijer;
GO

CREATE TABLE Cars (
    CarID INT,
    Make VARCHAR(50),
    Model VARCHAR(50),
    Year INT,
    Price INT
);
GO

--Insert routine
DECLARE @CarID INT = 1
DECLARE @Year INT = 2015
DECLARE @Price INT = 20000

WHILE @CarID <= 10
BEGIN

    INSERT INTO Cars
    VALUES (
        @CarID,
        'Ford',
        CONCAT('Model', @CarID),
        @Year,
        @Price
    )

    SET @CarID = @CarID + 1
    SET @Year = @Year + 1
    SET @Price = @Price + 1500

END
GO
SELECT * FROM Cars;
```

```
Msg 2714, Level 16, State 6, Line 7
There is already an object named 'Cars' in the database.

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(1 row affected)

(10 rows affected)

Completion time: 2024-04-04T20:30:44.1501376-04:00
```

Comments: This query creates a Cars table and inserts ten records using variables that increment for each row. The goal is to demonstrate how SQL can generate multiple records dynamically using loops and variable changes**.**

Query 16

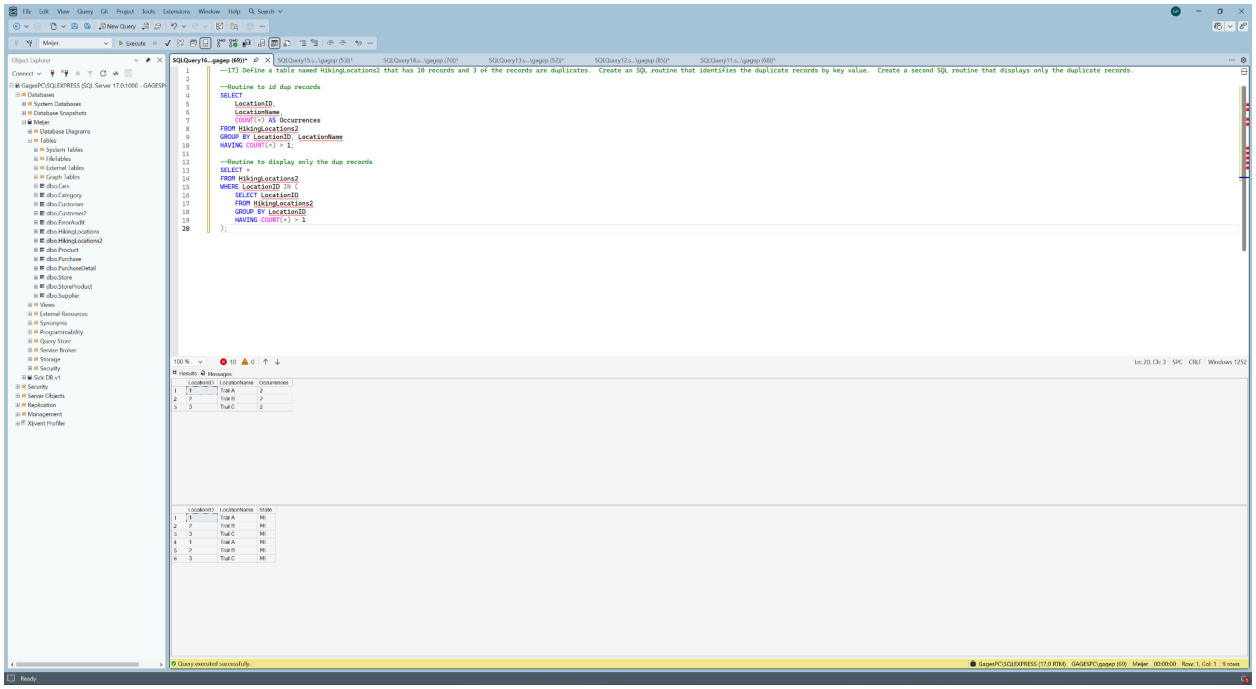


```
--16) Create a table named HikingLocations that has the appropriate structure to remove duplicates when loading the table.  Demonstrate that it works. Include sql that defines the structure to remove duplicates in your submission.

USE Meijer;
GO

CREATE TABLE HikingLocations (LocationID INT,LocationName VARCHAR(50),State VARCHAR(50));
GO

CREATE UNIQUE INDEX idx_HikingLocations
ON HikingLocations(LocationID)
WITH (IGNORE_DUP_KEY = ON);
GO

--Demonstrate it works
INSERT INTO HikingLocations VALUES (1, 'Sleeping Bear Dunes', 'Michigan')
INSERT INTO HikingLocations VALUES (2, 'Pictured Rocks', 'Michigan')
INSERT INTO HikingLocations VALUES (1, 'Sleeping Bear Dunes', 'Michigan')
GO

SELECT * FROM HikingLocations;
```

	LocationID	LocationName	State
1	1	Sleeping Bear Dunes	Michigan
2	2	Pictured Rocks	Michigan

Comments: This query creates the HikingLocations table with a unique index and the IGNORE_DUP_KEY option to prevent duplicate records from being inserted. The purpose is to demonstrate how SQL can enforce data integrity and automatically remove duplicate entries.

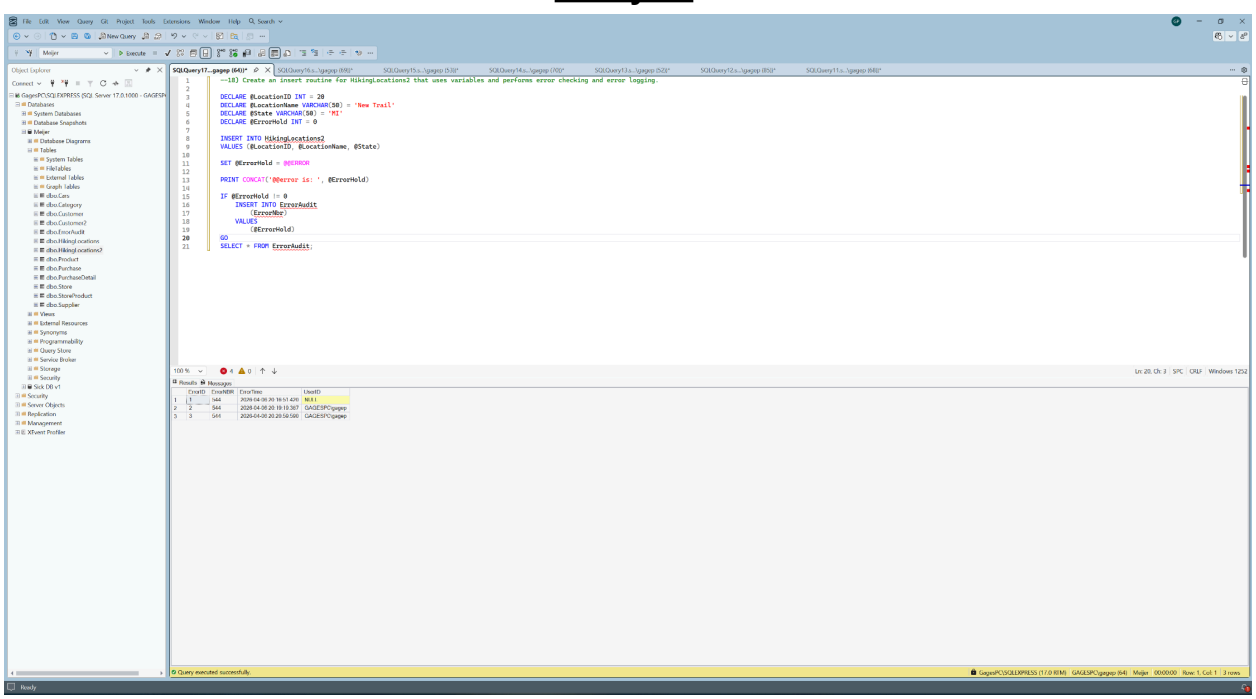
Query 17



```
--17) Define a table named HikingLocations2 that has 10 records and 3 of the records are duplicates.  Create an SQL routine that identifies the duplicate records by key value.  Create a second SQL routine that displays only the duplicate records.

--Routine to id dup records
SELECT
    LocationID,
    LocationName,
    COUNT(*) AS Occurrences
FROM HikingLocations2
GROUP BY LocationID, LocationName
HAVING COUNT(*) > 1;

--Routine to display only the dup records
SELECT *
FROM HikingLocations2
WHERE LocationID IN (
    SELECT LocationID
    FROM HikingLocations2
    GROUP BY LocationID
    HAVING COUNT(*) > 1
);
```

Comments: This query creates a HikingLocations2 table with duplicate records and uses SQL routines to identify and display duplicate entries. The goal is to demonstrate how SQL can detect and isolate duplicate data using grouping and counting techniques.

Query 18



```
--18) Create an insert routine for HikingLocations2 that uses variables and performs error checking and error logging.

DECLARE @LocationID INT = 20
DECLARE @LocationName VARCHAR(50) = 'New Trail'
DECLARE @State VARCHAR(50) = 'MI'
DECLARE @ErrorHold INT = 0

INSERT INTO HikingLocations2
VALUES (@LocationID, @LocationName, @State)

SET @ErrorHold = @@ERROR

PRINT CONCAT('@@error is: ', @ErrorHold)

IF @ErrorHold != 0
    INSERT INTO ErrorAudit
        (ErrorNbr)
    VALUES
        (@ErrorHold)
GO
SELECT * FROM ErrorAudit;
```

Comments: This query creates an insert routine for HikingLocations2 that uses variables with error checking and error logging. The purpose is to demonstrate how SQL routines can safely insert data while tracking any errors that occur.